

Skill Generalizer



`skill-generalizer` uses a teacher model to test whether fresh target models actually perform a skill as intended, then turns only confirmed failures into small, model-neutral lessons.

The central loop is:

- teacher writes paired probes + sealed retest
- fresh students answer the unchanged skill
- teacher grades and another grader audits
- one confirmed gap becomes one local lesson
- fresh source/candidate students take the retest
- accept only if every target passes without regression



[Watch the higher-quality 10-second MP4.](#)

Instruction length is not treated as a model property. Adding detail is only one possible patch; the system may instead sharpen a boundary, remove a distracting instruction, move a check, add a schema, or replace prose with deterministic validation.

The teacher does not ask a model what prompt it wants. It asks behavior-revealing questions and grades observable decisions, actions, claims, and outputs against a key written in advance.

The teacher generates the diagnostic, contrast, and retest scenarios from the source skill. It must generate the complete packet before student answers exist, then freeze it for that round. A small fixed regression set supports version-to-version comparison; a freshly generated sealed set checks that the patch did not merely memorize the benchmark. See [scenario design](#).

What users receive

Successful fitting produces one clean common `SKILL.md`, not separate Sol, Terra, and Luna prompts. Model names, questions, scores, and grading stay in an evidence bundle outside the published skill. The public skill contains the original intent plus only the smallest lessons that passed every target without regression.

See the [common skill output example](#) for a concrete public `SKILL.md` and its separate evidence layout. When a candidate does not pass—as in the current teacher–student pilot—the source remains the publishable skill and the candidate is retained only as unvalidated evidence.

Teacher–student pilot

The simple loop was exercised end-to-end in the [teacher–student pilot](#). Every agent used low reasoning.

Model	Diagnostic source	Retest source	Retest candidate	Target item
Sol	6/6	5/6	6/6	fail → pass
Terra	5/6	5/6	6/6	fail → pass
Luna	5/6	5/6	4/6	fail → fail

The added rule improved the target behavior from 0/3 to 2/3, but Luna still missed it and regressed on another item. Therefore the candidate is recorded as **unvalidated**, not fitted. That negative result is the point of the retest gate: an intuitively good lesson and a higher total score are not enough.

How fit is decided

Before editing, every expected behavior receives:

- a stable contract ID;
- observable calibration and contrast cases;
- a grader;
- a critical/non-critical designation;
- a pass threshold and repeat-aggregation rule;
- a frozen model, context, tool, and harness configuration.

A target fits the skill on the declared evaluation batch only when every critical item and threshold passes without regressing previously correct contract-case cells. The executing model's claim that it understands the skill is not evidence of fit, and passing a finite batch is not a universal reliability claim.

See [the fit specification](#) and [micro-adjustment rules](#) for a heavier-weight version.

Earlier behavior-fit pilot

The earlier contract-heavy method was exercised end-to-end in [behavior-fit experiment v2](#):

Model	Development source	Development patched	Hidden source	Hidden patched
Sol	18/18	18/18	12/12	12/12
Terra	18/18	18/18	12/12	12/12

Model	Development source	Development patched	Hidden source	Hidden patched
Luna	17/18	18/18	12/12	12/12

A single C2 precedence sentence repaired Luna's only observed development failure without a regression on Sol, Terra, contrast cases, or hidden cases. Since the source skill already passed every hidden case, v2 demonstrates counterexample repair and non-regression rather than hidden-set uplift. The experiment used one draw per case, so it does not establish repeated-run reliability.

Earlier density pilot

The repository retains an early minimal-change-router pilot under `benchmark/` and raw runs under `results/`. That pilot compared compact, balanced, and detailed whole-skill renderings:

Model	Compact calibration	Balanced calibration	Detailed calibration	Original hidden	Patched hidden
Sol	16/16	15/16	16/16	11/12	12/12
Terra	16/16	16/16	16/16	11/12	12/12
Luna	15/16	15/16	15/16	9/12	12/12

Its useful lesson was that whole-prompt density is the wrong abstraction: Luna's calibration error survived all three densities, while a localized boundary clarification worked in the hidden run. The pilot has important limitations—one batched run per condition and a post-hoc key clarification—so it is retained as motivation and raw evidence, not as proof of general reliability.

Layout

SKILL.md	behavior-fit workflow
references/fit-spec.md	acceptance-contract template
references/micro-adjustments.md	local patch operators and ledger schema
references/scenario-design.md	generated and fixed case protocol
references/common-skill-output-example.md	public output example
experiment-v3-teacher/	teacher-student pilot and raw evidence
experiment-v2/	behavior-fit micro-patch experiment
benchmark/	retained first pilot
results/	raw pilot runs and audit trail

Validate the retained pilot

```
python3 benchmark/grade.py path/to/answers.json benchmark/calibration-key.json
```